

Clearspace CS 499

Word-for-word Recording Script

This replaces the detailed speaking guide. Read only the large SAY THIS box. The small ON SCREEN box tells you what to open or point at, but you do not say it aloud.

ON SCREEN (do not read this out loud)

Keep Clearspace, your code editor, and this script open. Speak slowly, point at the code while you describe it, and pause for two or three seconds when a new file appears.

0:00 to 1:30 - Start the video

ON SCREEN (do not read this out loud)

Clearspace open on a safe normal folder. Keep the sidebar, file list, and search box visible.

SAY THIS WORD FOR WORD

Hello. This is my Clearspace code review. Clearspace is a Windows file-management application that lets a user browse folders, search for files, perform file operations, and organize files with tags. In this video, I will review the current code in three categories: software engineering and design, algorithms and data structures, and database design. For each category, I will explain what the current code does, identify one area that can be improved, and explain the enhancement I plan to make. I will begin by showing the application from the user point of view, and then I will connect each feature to the code that supports it.

1:30 to 3:30 - Show normal file browsing

ON SCREEN (do not read this out loud)

Click one folder, select one ordinary file, and navigate once.

SAY THIS WORD FOR WORD

A user begins by choosing a folder and selecting a file. The application has to remember the current folder, the selected item, the items that should appear in the main panel, and the actions the user is allowed to take. For example, selecting a file can enable actions such as open, copy, move, rename, delete, view properties, or assign tags. In a WPF application, the window is called the view. The ViewModel supplies the information and commands that the view needs. In Clearspace, the ViewModel coordinates the user-interface state while services perform the detailed file-system work. This separation is important because the window should not contain all of the Windows shell code or all of the search logic itself.

3:30 to 5:00 - Show search and tags

ON SCREEN (do not read this out loud)

Run a simple search. If possible, show one tag or the tag menu.

SAY THIS WORD FOR WORD

Search and tags are two important features in Clearspace. Search helps the user locate files without opening folders one by one. Tags let a user group related files without changing where those files are stored on disk. One file can have more than one tag, and the same tag can belong to many files. Later in the video, search will lead into my algorithms and data-structures review, and tags will lead into my database review. Now that I have shown the main user experience, I will begin the software engineering and design category by opening the project structure.

5:00 to 7:00 - Software engineering: project structure

ON SCREEN (do not read this out loud)

Open Clearspace/ARCHITECTURE.md. Point to Native, Models, Services, Commands, and ViewModels.

SAY THIS WORD FOR WORD

The project is organized into several areas with different responsibilities. Native code contains Windows-specific calls. Models represent application data. Services contain reusable behavior, such as file operations, search, indexing, and tag storage. Commands represent actions the user can take, such as copying or deleting a file. ViewModels hold information needed by the user interface and coordinate the work between the interface and the services. This is a good starting design because it gives each type of code a clear home. It also makes the program easier to maintain because a change to Windows-specific behavior does not have to be made directly in the user interface. Next, I will show the MainViewModel, which is the central controller for much of the interface.

7:00 to 10:00 - Software engineering: MainViewModel

ON SCREEN (do not read this out loud)

Open MainViewModel.cs at RunTreeSearchAsync. Point to ResolveSearchRoots, the cancellation token, seen, FileIndexService.Covers, and DispatcherTimer.

SAY THIS WORD FOR WORD

This is the MainViewModel. It is the central controller for the screen because it coordinates navigation, folder loading, search, sidebar entries, tags, saved preferences, and status information. I am now looking at one method inside it called RunTreeSearchAsync. This method prepares and runs a search through the selected folder locations.

First, it gets the folders that should be searched by calling ResolveSearchRoots. Then it checks whether the user entered a search query and whether there are valid folders to search. If either one is missing, the method exits early so the application does not do unnecessary work.

Next, the code creates a cancellation token. Searching through folders can take time. If the user changes the search or navigates somewhere else, Clearspace can cancel the old search instead of continuing unnecessary work in the background.

The code then creates a HashSet called seen. This stores file paths that were already found, so the same file is not displayed more than once. The stopwatch tracks search time. The method also checks whether the in-memory file index already covers every folder being searched. If it does not, Clearspace marks that it is performing a deeper tree search. Finally, the DispatcherTimer updates results every 250 milliseconds. This lets the application publish results in groups instead of refreshing the screen for every single result, which helps the interface stay responsive.

This method shows the strength of MainViewModel: it coordinates the search workflow, cancellation, duplicate prevention, index coverage, and user-interface updates in one place. However, it also shows the limitation. One class has many reasons to change, which can make it harder to maintain and test.

10:00 to 12:30 - Software engineering: file operations

ON SCREEN (do not read this out loud)

Open Clearspace/Commands, then FileOperationService.cs. Point to Delete, Copy, Move, Rename, Run, and the Boolean return value.

SAY THIS WORD FOR WORD

I will now move from the MainViewModel to FileOperationService. The MainViewModel coordinates what the user wants to do, but this service performs the actual Windows file operations. This service is a wrapper around Windows Shell APIs. That keeps low-level Windows file-operation code out of the user interface.

The Delete method accepts a list of file paths, the owner window, and a setting that decides whether the deletion is permanent. If there are no paths, it returns false immediately. If the deletion is not permanent, the code uses the allow-undo flag, which lets Windows send the files to the Recycle Bin. Copy, Move, and Rename each pass their operation type, paths, destination, and flags into the shared Run method. That is a good design choice because the detailed setup code is not duplicated.

Inside Run, the code creates an SHFILEOPSTRUCT. This is the structure Windows requires before it performs a file operation. The helper named ToDoubleNullTerminated changes the list of file paths into the special null-separated format that the Windows Shell API expects. The service then calls SHFileOperationW. The current code returns true only when Windows reports success and the user did not cancel the operation.

The limitation is this Boolean result. A true-or-false answer does not explain why an operation failed. The user may have canceled it, Windows may have denied permission, a file may have been missing, or another Windows error may have occurred. These are different situations, but the method returns false for all of them. The ShowProperties and OpenWith methods use the same idea by calling another Windows API with the properties or open-with command.

12:30 to 15:00 - Finish software engineering

ON SCREEN (do not read this out loud)
Leave MainViewModel.cs or FileOperationService.cs open.

SAY THIS WORD FOR WORD

To summarize the software engineering and design category, Clearspace already has a strong foundation. It separates the user interface, commands, services, models, and Windows-specific code instead of putting all logic directly into the window. The MainViewModel coordinates important interface behavior, and FileOperationService keeps Windows shell work separate from the interface.

My planned enhancement will build on this design instead of replacing it. I will split the oversized MainViewModel into smaller focused responsibilities, including navigation coordination, search coordination, and sidebar state. I will replace simple Boolean file-operation results with a richer result object. That result will communicate whether an operation succeeded, was canceled, failed because of permissions, failed because a file was missing, or failed for another Windows reason. I will also add automated tests for search behavior, tag behavior, indexing edge cases, and file-operation failures. These changes will make Clearspace easier to maintain, easier to test, and more reliable when handling user files.

That concludes my software engineering and design review. I will now move to algorithms and data structures, beginning with the search experience.

15:00 to 16:30 - Algorithms: why search uses an index

ON SCREEN (do not read this out loud)
Return to Clearspace and run another simple search.

SAY THIS WORD FOR WORD

For the algorithms and data-structures category, I am returning to search. A user expects search to respond quickly even when a drive contains many files. Walking through every folder on disk every time the user types a search would be slow. Clearspace can use visible-folder results, an in-memory index, Windows Search, and a fallback directory crawl when needed. The important idea is that an index stores information that has already been collected so repeated searches can look through memory instead of walking the disk again. This is a tradeoff. The application uses memory and background work to build and maintain the index, but it can return repeated search results more quickly. Next, I will show the data structure that stores the indexed information.

16:30 to 19:30 - Algorithms: FileIndex

ON SCREEN (do not read this out loud)

Open FileIndex.cs. Point to IndexEntry, NameOffset, ParentIndex, character pools, folded names, and Search.

SAY THIS WORD FOR WORD

This file contains the in-memory data structure used for indexed search. Each IndexEntry stores information about a file, including its size, timestamps, attributes, a name offset, and a parent index. Instead of storing a full separate name string inside every entry, Clearspace stores file names in a shared character pool. The name offset tells the application where a file name starts in that pool. This saves memory when the application indexes a large number of files.

ParentIndex records the relationship between an item and its containing folder. When a result needs a full path, the application can follow those parent relationships instead of storing a complete repeated path in every entry. The folded-name pool keeps a normalized lowercase version of names. That means the application does not need to repeatedly convert every file name to lowercase every time the user changes the search query.

The Search method looks through these entries, applies the user's search filters, and can scan independent chunks in parallel. The tradeoff is clear: Clearspace uses more memory and background indexing work, but it can make repeated searches faster. The next file shows how this index is built by scanning the file system.

Minutes 19 to 22 - FileIndexBuilder

ON SCREEN (do not read this out loud)

Open FileIndexBuilder.cs. Point to MaxDepth = 32, the pending stack, and the reparse-point check.

SAY THIS WORD FOR WORD

FileIndexBuilder creates the in-memory index by walking directories in the background. It uses a stack to track directories that still need to be scanned. A stack supports a depth-first traversal. The builder takes one pending directory, records its files and folders, and adds safe child directories to the list of work that remains.

The code also avoids reparsing points. This is important because symbolic links and Windows junctions can create cycles or lead the search outside the intended directory tree. Skipping them is a defensive choice that helps prevent the index builder from getting stuck in a loop.

However, there is a limitation near the top of the file. MaxDepth is set to 32. This means a file that is nested more than 32 folder levels below the starting location may not be added to the index. That can become a correctness problem because a user may expect search to include all files. The next fallback search path has a second depth limit, so I will compare it next.

22:00 to 24:00 - Algorithms: fallback search

ON SCREEN (do not read this out loud)

Open FileSearchService.cs. Point to MaxDepth = 24, worker tasks, queue, and batched results.

SAY THIS WORD FOR WORD

FileSearchService is the fallback search path for locations that are not completely covered by the in-memory index. It uses background workers to scan directories and publishes results in batches. This is useful because the user can begin seeing matches while additional folders are still being scanned.

The concern is that this fallback search has another hard-coded depth limit. In this file, MaxDepth is 24, which is different from the index builder limit of 32. That means the fallback search can include fewer nested folders than the main index. A user could receive different search coverage depending on which search path is used. The code is trying to be responsive, but the two different depth caps create a correctness and consistency concern.

My enhancement will make traversal depth configurable or remove arbitrary caps after I verify that cycle protection and cancellation remain safe. I will test both search paths with the same deep folder structures so the application gives more consistent results.

24:00 to 26:00 - Finish algorithms and data structures

ON SCREEN (do not read this out loud)
Return to FileIndex.cs and point to ParentIndex again.

SAY THIS WORD FOR WORD

My algorithms and data-structures enhancement has two parts. First, I will make traversal depth configurable or remove the arbitrary depth limits while keeping the safe handling of reparse points and cancellation. I will test deep folder structures, renamed files, deleted files, and canceled searches in both search paths.

Second, I will add folder-size aggregation to the index. Each indexed item already stores its size and its parent relationship. After an index build, Clearspace can work from the deepest items upward. Each file adds its size to its parent folder, and each folder adds its completed total to its own parent. This is called a bottom-up pass. It lets Clearspace show the total size of a folder without scanning the disk again every time the user asks for it.

I will measure memory use and indexing time, and I will add correctness tests for nested folders. The goal is not only fast search. The goal is search and folder information that users can trust. That concludes my algorithms and data-structures review. I will now move to the database category by returning to tags.

26:00 to 27:30 - Database: explain tags

ON SCREEN (do not read this out loud)

Return to Clearspace. Show a tag, tag menu, or tag-related search filter.

SAY THIS WORD FOR WORD

For the database category, I am returning to tags. Tags are user-managed metadata. A file can have multiple tags, and one tag can be used on many files. For example, one file could have both a School tag and a Portfolio tag, while the School tag can be attached to many different files. This is called a many-to-many relationship.

The user experience is simple, but the application has to store the relationship accurately as tags are added, removed, renamed, and deleted. The question for this category is whether the current storage design remains reliable as the user creates more tags and assignments over time. Next, I will show the current tag storage service.

27:30 to 28:00 - Open TagService

ON SCREEN (do not read this out loud)
Open TagService.cs and pause at the top of the file.

SAY THIS WORD FOR WORD

The tag feature is simple for the user, but the next file shows how Clearspace stores the tag definitions and file assignments behind the screen. I will now look at the current storage design and explain why I plan to improve it.

TagService

ON SCREEN (do not read this out loud): Open TagService.cs. Point to TagData, Definitions, Assignments, Load, Save, and Delete.

SAY THIS WORD FOR WORD

This service shows how tags are stored today. TagData contains a list of tag definitions and a dictionary that maps a file path to a list of tag IDs. The definitions represent the available tags. The assignment dictionary records which tag identifiers belong to each file path. This correctly represents the basic many-to-many relationship in application memory, and JSON is simple to use for a small amount of data.

However, each update rewrites the full JSON document. When a tag is deleted, the application manually searches assignments to remove tag IDs that no longer belong to a valid tag. This means relationship cleanup is handled by application code. It also means related changes are not protected by a database transaction. Data integrity means the stored data stays consistent. In this case, an assignment should not point to a tag that no longer exists. The current JSON approach is simple, but it becomes harder to manage as tags and assignments grow.

29:00 to 30:00 - Database enhancement and final close

ON SCREEN (do not read this out loud)
Leave TagService.cs open.

SAY THIS WORD FOR WORD

My planned database enhancement is a local SQLite database for tag metadata. I will use a Tags table for tag definitions and a FileTags table for assignments. The FileTags table will store the connection between a file path and a tag ID, which is the relational form of the many-to-many relationship.

I will enable foreign keys so an assignment cannot refer to a tag that does not exist. I will add an index for tag lookups, use parameterized queries, and perform related changes inside transactions. I will also migrate existing JSON data safely and keep a backup until the migration succeeds.

To conclude, Clearspace already has a strong foundation. My planned enhancements will make the application easier to maintain, more accurate when searching, and more reliable when storing tag data. Thank you for reviewing my code-review plan.